Design Documentation

Interactive House - AI Smart Control System

Revision History

Date	Version	Description	Author
2026-02-24	1.0	Initial AI design including safety and sensors (D1, D2)	Anitta Antony
2026-02-24	1.0	Initial AI design including safety and sensors (D3, D4)	Md Abu Taher
2026-02-28	1.0	Smart Home code	Daniel Marcarini
2026-02-28	1.0	Speech Recognition Initial Design D5	Daniel Holderik
2026-03-15	1,0	Floating Speech control and revission	Daniel Holderik
2026 -03-16	1.1	Modify D1-D4 for Web	Anitta Antony Md Abu Taher
2026-03-16	1.1	Testing	Daniel Marcarini
2026-04-24	1.2	Initial Speech Recognition for web (D5)	Daniel Holderik

Design item List

Requirement Name	Priority	Progress
D1. AI Architecture Overview & Component Design	Essential	Done
D2. Intent Output Format + Prompting Rules(Firebase AI)	Essential	Done
D3. AI chat UI Design (Mobile+Web) + API Contract	Essential	Done
D4. Error Handling, Reliability & State/Context Handling	Desirable	Done
D5. Speech Recognition Integration and Voice Navigation Design	Essential	
D6. Web application testing strategy for device-control integration	Desirable	

Design Item Descriptions

D1 - AI Architecture Overview & Component Design

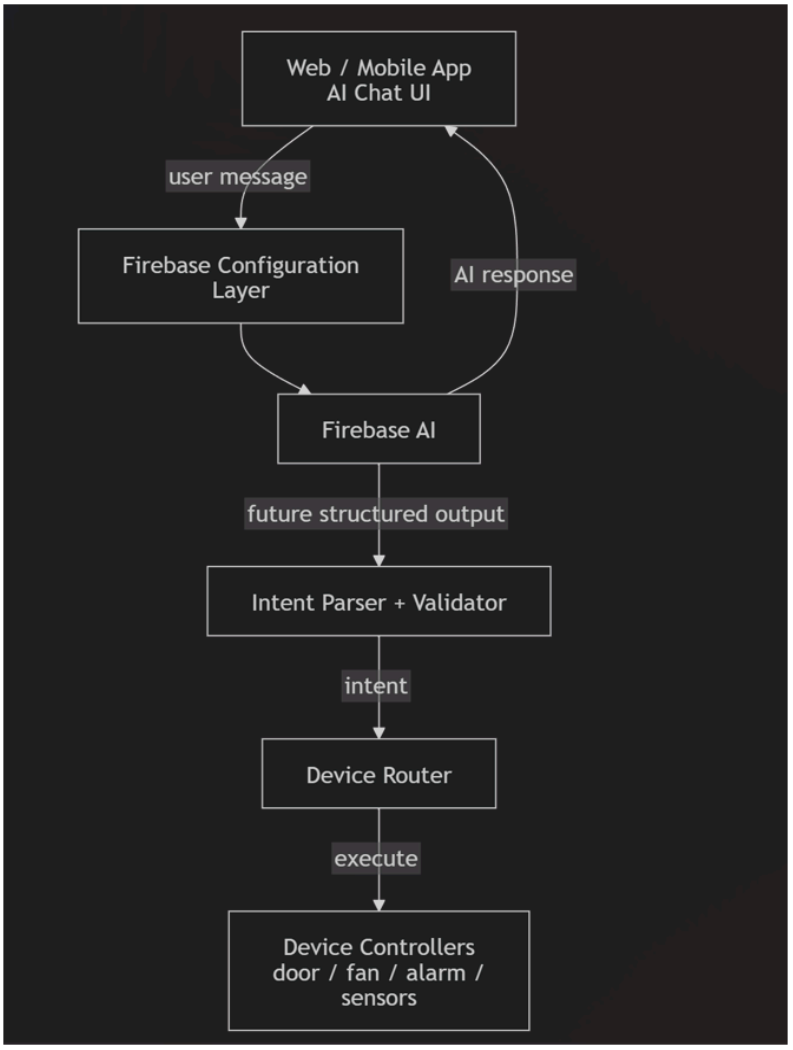
The AI feature is implemented for both web (Next.js) and mobile (React Native) using a shared Firebase configuration.

User messages are sent from the chat interface to Firebase AI, which generates responses and returns them to the UI.

The web client sends messages using direct AI requests, while the mobile client maintains a chat session for continuous conversation.

The UI displays the response and keeps chat history visible.

Figure 1:AI Architecture using Firebase AI for Web and Mobile Applications



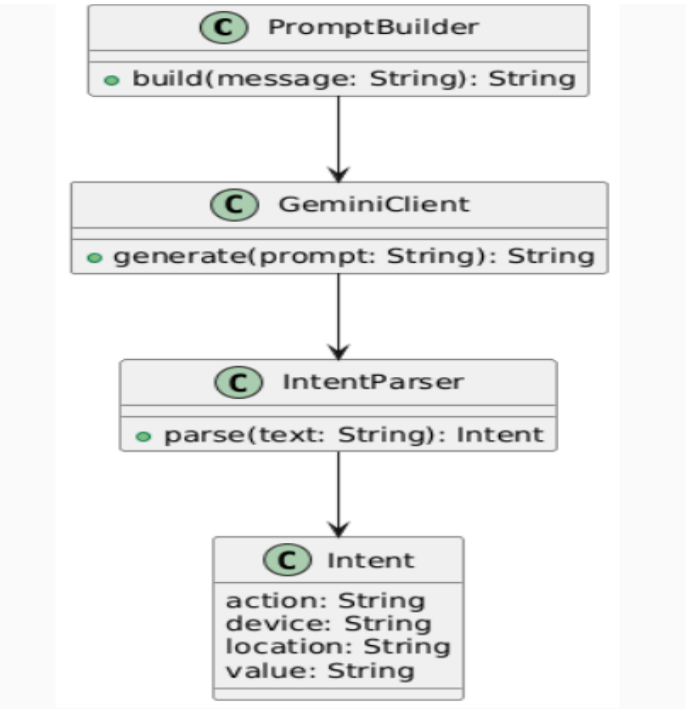
D2 - Intent Output Format + Prompting Rules (Firebase AI)

The AI system generates human-readable responses for the chat interface. Responses are used mainly for conversation, but the design also supports structured output for future smart-home commands.

Example intent JSON

{
"action": "open",
"device": "door",
"location": "main",
}
Using Firebase AI allows flexible responses while keeping the system ready for command interpretation.

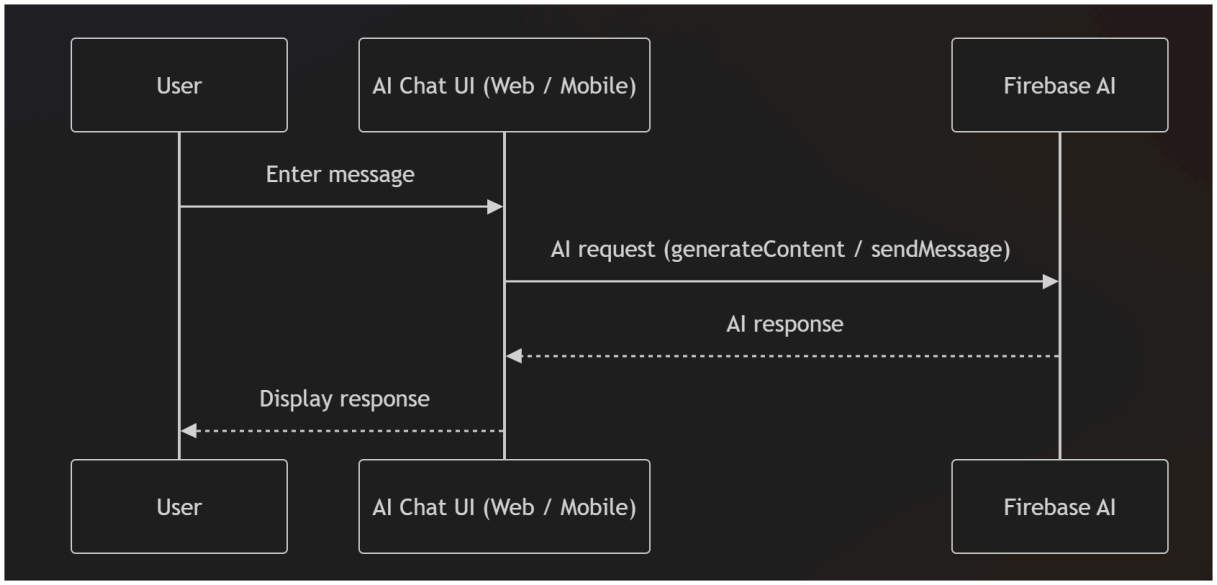
Figure – Design Class Diagram (AI intent pipeline)



D3 - Mobile AI Chat UI Design (AiScreen.tsx) + API Contract

The AI what interface is implemented in both the web and mobile applications. The UI keeps message history, allows user input, shows loading state, and displays AI responses.Web version uses generateContent() for each message.Mobile version uses a chat session to keep conversation context.Both versions use the same Firebase configuration.
This provides consistent behaviour across platforms

Figure – Sequence Diagram



D4 - Error Handling and Frontend State Management

Description

This design item explains how the frontend ensures reliability and safe interaction with the AI backend.

Reliability Rules

- Prevent empty message submission
- Prevent duplicate requests while loading
- Display configuration error if Firebase configuration is missing
- Show fallback message if AI request fails

Error Handling

- **Missing backend configuration:** Display setup warning
- **Network Error:** Show network error message
- **Invalid JSON response:** Handle safely without crashing
- **Non-200 response:** Display returned error message

State Management

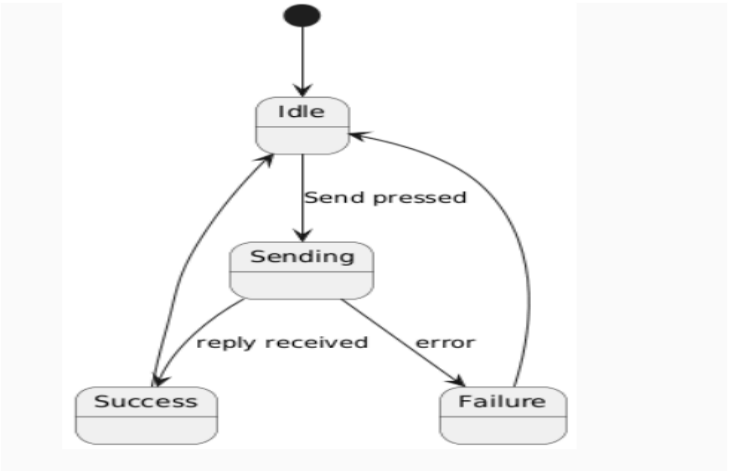
The frontend manages request states using a loading flag:

- Idle
- Sending
- Success
- Failure

This allows the AI system to correctly interpret context-dependent commands such as “Turn it off” by identifying which device is being referenced.

Maintaining device states ensures consistent behavior and improves the intelligence and usability of the AI system.

Figure – State Diagram



D5 – Speech Recognition Integration and Voice Navigation Design

- **Description**

This design item describes the integration of speech recognition into the mobile application using Expo and an external speech recognition library. The feature allows users to control the Interactive House system using spoken commands instead of manual text input.

- **Technology and Build Configuration**

The speech recognition functionality requires native device capabilities that are not available in the standard Expo Go environment. Therefore, a custom Expo development build was created to support the external speech recognition library and enable microphone access and real-time voice processing.

- **Architecture Integration**

The speech recognition module is implemented as a dedicated screen (tab) within the mobile application. The module interacts with the existing navigation and AI chat system.

The interaction flow is:

1. The user opens the Speech Recognition tab.

2. The user activates the microphone using the UI control.

3. Spoken input is captured and converted into text using the speech recognition library.

4. The recognized text is displayed and forwarded to the backend AI API. (Future feature)

5. The backend processes the command and returns a response.(Future feature)

The application navigates to the relevant screen or updates the system state based on the interpreted command.

- **Navigation Integration**

Speech recognition can trigger navigation between different screens in the application. This allows users to move between system features using voice commands, improving accessibility and interaction efficiency.

- **Global floating voice control interface**

The speech system was full redesigned and the dedicated speech screen was removed and replaced with a persistent floating voice control component.

The floating voice control icon stays attached across all screens and follows the user interface during navigation and scrolling. This makes sure that voice interaction is always available without requiring the user to manually open a specific feature tab.

- **Implementation**

The floating component is implemented at the application root level so that it remains active regardless of the currently displayed screen. The icon maintains a fixed overlay position above the main interface while allowing the user to continue interacting with the rest of the application.

- **Interaction Flow**

1. The user activates the floating microphone icon.

2. The application captures spoken input using the speech recognition module.

3. The speech input is converted into text commands.

4. The system interprets the command and triggers the corresponding UI interaction.

- **UI Control Integration**

The speech recognition system is connected to existing interface controls including:

• Door control buttons

• Window control buttons

• Ventilation and device toggles

• Sliders and adjustable controls

Voice commands trigger the same underlying functions used by the UI controls. This ensures consistent behavior between manual interaction and speech-based interaction.

D6 – Web application testing strategy for device-control integration

This design item describes how the web application will be verified during the current sprint. The testing focus is the interaction path between the web user interface, Firebase Authentication, Firestore, and the device state representation used by the project.

The verification strategy includes:

- manual testing of login and signup validation
- manual testing of navigation and guest access
- verification of Firestore-backed device toggles

- verification of real-time synchronization through snapshot updates
- defect logging and retesting after fixes

This strategy supports reliable integration between subgroup components and reduces the risk of hidden faults in the device-control workflow.

Web Speech API Integration and Global State Management

The speech recognition system was updated to use the browser based Web API. This allows speech processing to be handled directly within the web environment without requiring a custom development build.

- Technology Change

The Web Speech API provides real time speech to text functionality within supported browsers. This removes the dependency on native modules and simplifies deployment for web based environments.

- Global State Integration

Speech recognition is integrated with the application's global state management system. This enables voice commands to be processed and executed consistently across all screens without requiring local component level handling.

The global state acts as a central controller for voice input, allowing commands to trigger UI actions regardless of the current screen or component.

- Interaction Flow (Updated)

1. The user activates the floating voice control icon.

2. The Web Speech API captures and processes spoken input.

3. The recognized text is stored in global state.

4. The application interprets the command and triggers the corresponding UI action.

5. UI components react to state changes and execute actions such as button presses or slider updates.

Advantages

• No dependency on native speech libraries

• Simplified deployment for web platforms

• Centralized voice control through global state

• Improved consistency across multi-screen interaction

Daniel Marcarini - Smart Home code

Requirement Name	Priority
D1. Embedded system architecture and hardware integration	Essential
D2. Sensor monitoring and state machine design	Essential
D3. Actuator control and alarm management system	Essential
D4. LCD display and user interaction handling	Essential
D5. Non-blocking timing and concurrent operation design	Essential

D1 – Embedded System Architecture & Hardware Integration

This design item defines the overall architecture of the embedded Smart Home system and the integration of hardware components.

The system is built around the Arduino Uno microcontroller, which serves as the central processing unit. All sensors, actuators, and interface components are connected directly to the microcontroller.

The architecture separates system functionality into three logical layers:

- Input Layer (gas sensor, temperature sensor, LDR, push buttons, Bluetooth)
- Processing Layer (state management and decision logic)
- Output Layer (servo motors, buzzer, LEDs, LCD display)

This layered design improves clarity and modularity while maintaining simplicity suitable for embedded systems.

D2 – Sensor Monitoring and State Machine Design

This design item specifies how environmental data is processed and how system states are managed.

The system continuously monitors:

- Gas levels
- Temperature
- Ambient light

Gas detection is implemented using a threshold-based comparison combined with a staged state machine. The state machine ensures that safety actions are executed in a controlled sequence rather than simultaneously.

Internal state variables maintain:

- Window open/closed state
- Door open/closed state
- Alarm activation status
- Gas detection transition state

This prevents repeated triggering and ensures stable system behavior.

D3 – Actuator Control and Alarm Management System

This design item describes how mechanical and alarm components are controlled.

The system controls:

- Window servo motor
- Door servo motor
- Buzzer (multiple modes)
- LED indicators

Buzzer behavior is centralized using defined operational modes (e.g., off, solid alarm, siren mode). This prevents conflicting sound activation from different subsystems.

During emergency events:

- The buzzer is activated
- The window may open automatically
- Visual indicators reflect hazard status

Safety-related actions override normal user interaction.

D4 – LCD Display and User Interaction Handling

This design item defines how the system provides feedback and processes user input.

The LCD module displays:

- Environmental readings
- Alarm notifications
- Device status information

Display updates are controlled to prevent flickering and unnecessary refresh operations.

User interaction is implemented using:

- Push buttons with edge-detection logic
- Bluetooth communication for remote commands

Button presses are detected through state comparison to ensure reliable toggling behavior.

D5 – Non-Blocking Timing and Concurrent Operation Design

This design item explains how the system maintains real-time responsiveness.

The system uses non-blocking timing mechanisms based on internal timers instead of delay-based blocking. This allows:

- Continuous sensor monitoring
- Immediate emergency response
- Simultaneous user interaction handling
- Stable LCD updates

This design ensures that no subsystem prevents another from executing, maintaining reliable concurrent behavior within the embedded environment.